CSC 212: Data Structures and Abstractions

08: Priority Queues, Binary Heaps

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Spring 2026

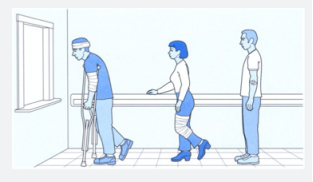
THINK BIG WE DO℠

Priority queues

Priority queues

- Definition
  - a **priority queue** is a data structure similar to a queue, but where each element has an associated priority
  - elements with **higher priority** are removed before elements with lower priority
- Operations
  - mutators
    - **enqueue**: (**push**) add element with an associated priority
    - **dequeue**: (**pop**) remove element with highest priority
  - accessors
    - **top**: returns a reference to the top (first) element
- Applications
  - algorithms for graphs, event-driven simulation, search methods in AI, job scheduling in OS, etc.

core push/pop **priority queue** operations run in $\Theta(\log n)$ time (assuming efficient implementations)



<key,value> pairs

- Elements in a priority queue can be implemented as a collection of **<key,value>** pairs
  - **key**: determines the priority, used for comparison
  - **value**: actual data/payload associated with that priority

Operations	Return value
enqueue(5, A)	
enqueue(10, D)	
top()	(5, A)
enqueue(3, B)	
dequeue()	
enqueue(7, C)	
dequeue()	
top()	(7, C)
dequeue()	
size()	1
isEmpty()	FALSE

min-priority queue example (smallest key has higher priority)

Practice

- What is the output of the following code?

```
#include <iostream>
#include <queue>

int main() {
    // by default, std::priority_queue is a max-priority queue
    std::priority_queue<std::pair<int, std::string>> pq;

    pq.push({3, “New York”});
    pq.push({9, "Boston"});
    pq.push({5, "Chicago"});

    while (!pq.empty()) {
        std::cout << pq.top().first << ": " << pq.top().second << "\n";
        pq.pop();
    }

    return 0;
}
```


Practice

- What is the output of this code?

```
#include <iostream>
#include <queue>
#include <utility> // for std::pair

int main() {
    // default priority_queue - max-priority queue behavior
    std::priority_queue<std::pair<int, std::string>> pq;

    pq.push({3, "Job 1"});
    pq.push({1, "Job 2"});
    pq.push({5, "Job 3"});
    pq.pop();
    pq.push({2, "Job 4"});
    pq.pop();
    pq.push({7, "Job 5"});
    pq.pop();
    pq.pop();
    pq.push({7, "Job 6"});
    pq.push({7, "Job 7"});

    while (! pq.empty()) {
        std::pair<int, std::string> top = pq.top();
        std::cout << top.second << std::endl;
        pq.pop();
    }

    return 0;
}
```

std::pair comparison is lexicographic, so among equal integer keys the string is the tiebreaker

Implementation

- Using arrays
  - ✓ array can be <u>fixed-length</u> or a <u>dynamic array</u>
- Considerations
  - ✓ highest priority
    - in a **max-priority queue**, refers to largest key value
    - in a **min-priority queue**, refers to smallest key value
  - ✓ equal priority elements
    - in some implementations, these elements are served in FIFO order
    - in others, their order is undefined
  - ✓ throw an error when calling dequeue on an empty queue
  - ✓ throw an error when calling enqueue on a full queue

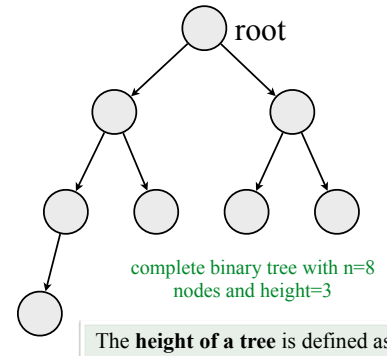
Implementation

- Array-based (keep an unsorted array)
  - ✓ <u>enqueue</u> at the end — $\Theta(1)$ cost (amortized cost if using a dynamic array)
  - ✓ <u>dequeue</u> (extract max/min) — $\Theta(n)$ cost
    - requires searching the entire array
- Array-based (keep a sorted array)
  - ✓ <u>enqueue</u> at position — $\Theta(n)$ cost
    - requires finding position for insertion and shifting elements
  - ✓ <u>dequeue</u> (extract max/min) — $\Theta(1)$ cost
- Binary heap (array-based)
  - ✓ most common and efficient
  - ✓ <u>enqueue</u> — $\Theta(\log n)$ cost
  - ✓ <u>dequeue</u> (extract max/min) — $\Theta(\log n)$ cost
  - ✓ can also build a binary heap from an unsorted array in $\Theta(n)$ cost (heapify)

Complete binary trees

Complete binary tree

- Binary tree
  - ✓ tree data structure in which each **node** has at most two children, referred to as the left child and the right child
- Complete binary tree
  - ✓ binary tree in which every level, except possibly the last, is completely filled
    - all nodes in the last level are as far left as possible



complete binary tree with n=8 nodes and height=3

The **height of a tree** is defined as the number of edges on the longest path from the root to a leaf (terminal) node.

`The height of a complete binary tree with` n `nodes is` $\lfloor \log_2 n \rfloor$

Practice

- Consider a complete binary tree of height h
  - ✓ what is n_{max}, the max number of nodes in the tree as a function of h?
    - hint: use a summation formula
  - ✓ what is n_{min}, the min number of nodes in the tree as a function of h?

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$
$$\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2}$$
$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$
$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2 = \frac{n^2(n+1)^2}{4}$$
$$\sum_{i=0}^{n} c^i = \frac{c^{n+1}-1}{c-1}, c \neq 1$$

- For a complete binary tree of height h, the following holds: $n_{min} \leq n \leq n_{max}$
  - ✓ for simplicity, we can also use $n_{min} \leq n < n_{max} + 1$
  - ✓ take the logarithm (base 2) of this inequality and express h in terms of n

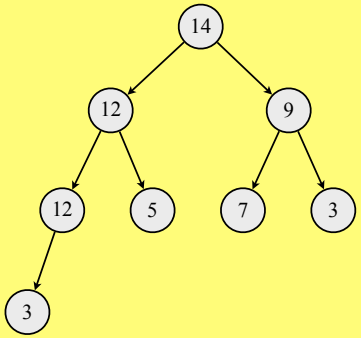
Binary heaps

Binary heap

- Definition
  - ✓ **structure property**: a binary heap is a **complete binary tree**
  - ✓ **heap property**: a binary heap can be:
    - **max-heap**: each node's value is greater than or equal to its children's values
    - **min-heap**: each node's value is smaller than or equal to its children's values
- Considerations
  - ✓ the height of a binary heap is $\lfloor \log_2 n \rfloor$
  - ✓ the number of nodes at each level h is at most 2^h
  - ✓ the number of nodes in a heap is at most: $\sum_{i=0}^{h} 2^i = 2^{h+1} - 1$

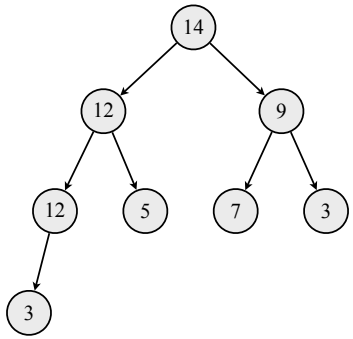
Practice

- Check:
  - ✓ structure property
  - ✓ heap-order property (max-heap)
- Add 3 elements at lowest level
  - ✓ without violating properties
- Change 2 values
  - ✓ that violate the heap property



Array representation

- A binary heap can be represented as an array
  - ✓ **root** is at index 0
  - ✓ **last element** is at index $n - 1$
- For any node at index i:
  - ✓ **left child** is at index $2i + 1$
  - ✓ **right child** is at index $2i + 2$
  - ✓ **parent** is at index $(i - 1) / 2$ (integer division)



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
14	12	9	12	5	7	3	3								

`n=8, capacity=16`